

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

REPORT

Laboratory work No. 8

System Calls and Inter-Process Communication in xv6

Rotaru Anastasia, FAF-242

Course: **Operating Systems**

Checked by: Patricia Reiţman, asist. univ.

Chişinău, 2026

Contents

1 Introduction **2**

2 Objectives **2**

3 Task 1: Process Execution inside xv6 **2**
3.1 Source code 2
3.2 Execution output 3
3.3 Observations 4

4 Task 2: Bidirectional IPC (Ping-Pong) **5**
4.1 Source code 5
4.2 Execution output 6
4.3 Observations 6

5 Optional: primes **6**

6 Conclusions **7**

1 Introduction

The aim of this laboratory work is to run the process system calls inside xv6, a small educational operating system for RISC-V, and to build a bidirectional communication between two processes with pipes. The first task repeats the `fork`, `exec` and `wait` sequence of the previous laboratory work inside xv6. The second task implements a ping-pong exchange of one byte between a parent and a child process.

All programs were built with `riscv64-linux-gnu-gcc` 13.3.0 and run in QEMU 8.2.2 (`make qemu`) on Ubuntu 24.04. Each new program was added to xv6 as a file `user/NAME.c` and a line `$U/_NAME\` in the `UPROGS` list of the `Makefile`:

```
UPROGS=\
        $U/_cat\
        ...
        $U/_run\
        $U/_pingpong\
        $U/_primes\
        ...
```

2 Objectives

The objectives of this laboratory work are the following. The first objective is to run a program inside the xv6 operating system that creates a child process with `fork()`, replaces the child with another program with `exec()`, and waits for it with `wait()`. The second objective is to compare these calls with the corresponding calls used on Linux in the previous laboratory work, and to explain the meaning of the exit status. The third objective is to build a bidirectional exchange of one byte between a parent and a child process, using two pipes, and to explain why two pipes are needed and which call makes the parent wait for the child.

3 Task 1: Process Execution inside xv6

The program `user/run.c` performs the three steps of the shell for the command `echo hello from the child`. The process creates a child with `fork()`, the child becomes `echo` with `exec()`, and the parent waits for the child with `wait()`. The program uses only the system calls of the xv6 kernel, because xv6 has no C library based on glibc.

3.1 Source code

Listing 1: Source code of `user/run.c`

```c
#include "kernel/types.h"
#include "user/user.h"

// the three steps of the shell, inside xv6:  echo hello from the child

int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());
```

```
10
11    int pid = fork();                    // step 1: copy myself
12    if(pid < 0){
13      printf("fork failed\n");
14      exit(1);
15    }
16
17    if(pid == 0){                        // only the child runs this
18      printf("child: my pid is %d\n", getpid());
19      char *args[] = { "echo", "hello", "from", "the", "child", 0 };
20      exec("echo", args);                // step 2: become echo
21      printf("exec failed\n");           // runs only if exec failed
22      exit(1);
23    }
24
25    int status;
26    wait(&status);                       // step 3: wait for the child
27    printf("parent: child %d ended with status %d\n", pid, status);
28    exit(0);
29  }
```

3.2 Execution output

Child runs `echo`. After `make qemu`, the program was started from the xv6 shell:

Listing 2: Output of `run` inside xv6 (child becomes `echo`)

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
$
```

The shell `sh` is process 2 (process 1 is `init`), so `run` receives PID 3 and its child PID 4.

Child runs `ls`. The two lines of the child were changed so that it becomes `ls`:

```
    char *args[] = { "ls", 0 };
    exec("ls", args);                    // step 2: become ls
```

Listing 3: Output of `run` with the child running `ls`

```
$ run
parent: my pid is 3
child: my pid is 4
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 35424
```

```
echo            2 4 34352
sleep           2 5 34072
args            2 6 34072
mystat          2 7 35112
find            2 8 41608
run             2 9 34656
pingpong        2 10 34688
primes          2 11 35424
forktest        2 12 17328
grep            2 13 42784
init            2 14 34816
kill            2 15 34280
ln              2 16 34088
ls              2 17 41568
mkdir           2 18 34328
rm              2 19 34312
sh              2 20 56176
stressfs        2 21 35208
usertests       2 22 200104
grind           2 23 50552
wc              2 24 36384
zombie          2 25 33696
logstress       2 26 36232
forphan         2 27 35096
dorphan         2 28 34544
sync            2 29 33760
console         3 30 0
parent: child 4 ended with status 0
$
```

The child prints the root directory of the xv6 file system. Each line shows the name, the type (1 = directory, 2 = file, 3 = device), the inode number and the size in bytes. The programs `run`, `pingpong` and `primes` are present because of the `UPROGS` lines.

Child runs a program that does not exist. The call to `exec` was changed to `exec("nosuchprog", args);`:

Listing 4: Output of `run` with a nonexistent program

```
$ run
parent: my pid is 3
child: my pid is 4
exec failed
parent: child 4 ended with status 1
$
```

Here `exec` fails and returns, so the child continues with the lines after it: it prints `exec failed` and calls `exit(1)`. The handout shows `child 8` because there the program was run after several other commands in the same session, and every new process gets the next PID; after a fresh boot the child is PID 4.

3.3 Observations

Question 1. Which calls does `run.c` use in xv6, and which did the Linux version use? The xv6 program uses `fork()`, `exec()` and `wait()`. The Linux program of the

previous laboratory work used `fork()`, `execlp()` and `waitpid()`. The three operations are the same: the process copies itself, the copy is replaced by another program, and the original process waits for the copy to end. The differences are only in the details. The xv6 `exec` takes the program name and the argument array and does not search `PATH`, and `wait` takes a pointer to a status variable but no process identifier: it waits for any child and returns its PID.

Question 2. Who prints the text "hello from the child": the code of `run`, or `echo`? The text is printed by `echo`. The code of `run` only prepares the argument array and calls `exec()`. After the successful call, the child runs the code of `echo`, which writes its arguments to standard output. The rest of `run` is gone from the child, which is why `exec failed` is not printed in the first two runs.

Question 3. Why is the status 1 in xv6 and 127 on Linux? The status is decided by the argument of the `exit()` call that follows the failed `exec`. In the xv6 program this call is `exit(1)`, so the child reports the value 1. In the Linux program of the previous laboratory work the call was `exit(127)`, which is the conventional code for a missing command, and the parent extracted it with `WEXITSTATUS(status)`. xv6 stores the value passed to `exit` directly, so no macro is needed. The number is chosen by the programmer, not by the operating system.

4 Task 2: Bidirectional IPC (Ping-Pong)

The program `user/pingpong.c` uses two pipes. The first pipe carries one byte from the parent to the child, and the second pipe carries one byte from the child back to the parent. The child prints that it has received the ping before it sends the byte back, and the parent prints that it has received the pong after it reads the byte.

4.1 Source code

Listing 5: Source code of `user/pingpong.c`

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];     // parent-to-child, child-to-parent
  char b;

  pipe(p2c);
  pipe(c2p);

  if(fork() == 0){
    close(p2c[1]); close(c2p[0]);       // the ends the child does not use
    read(p2c[0], &b, 1);                // read 1 byte from the parent
    printf("%d: received ping\n", getpid());
    write(c2p[1], &b, 1);               // send the byte back
    exit(0);
  }

  close(p2c[0]); close(c2p[1]);         // the ends the parent does not use
```

```
  b = 'x';
  write(p2c[1], &b, 1);                 // send 1 byte to the child
  read(c2p[0], &b, 1);                  // wait for the reply
  printf("%d: received pong\n", getpid());
  wait(0);
  exit(0);
}
```

4.2 Execution output

Listing 6: Output of `pingpong` inside xv6

```
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
$ pingpong
6: received ping
5: received pong
$
```

The ping is printed first, by the child, and the pong second, by the parent. `pingpong` was started in the same session right after `run`, which had used PIDs 3 and 4, so the parent is PID 5 and the child PID 6.

4.3 Observations

Question 1. Why are two pipes needed, and not one for both directions? A pipe in xv6 and in Linux is unidirectional: data written to the write end can be read only from the read end. If both processes used one pipe, a process could read back the byte it had just written itself, and nothing would tell the two messages apart. Two pipes give each direction its own channel, so each process reads only the data sent to it.

Question 2. Nothing in the code says to wait until the child has replied. Which call makes the parent wait? The parent waits in the call `read(c2p[0], &b, 1)`. This call blocks until the child writes its byte or closes the write end of the second pipe. The final call `wait(0)` then waits until the child has exited.

5 Optional: primes

The program `user/primes.c` prints the primes from 2 to 35 with a chain of processes. Each stage reads its first number p, which is prime, prints it, creates a pipe and a child, and forwards to the child only the numbers not divisible by p. Every unused pipe end is closed, so each stage sees end-of-file when the previous one finishes.

Listing 7: Output of `primes` inside xv6

```
$ primes
prime 2
prime 3
prime 5
```

```
prime 7
prime 11
prime 13
prime 17
prime 19
prime 23
prime 29
prime 31
$
```

6 Conclusions

The laboratory work showed that the process system calls of xv6 have the same meaning as those of Linux. The sequence `fork()`, `exec()` and `wait()` runs a program in a child process, a failed `exec` returns to the old code, and the exit status is chosen by the program that calls `exit()`. The `pingpong` program showed that two pipes are needed for a two-way exchange, and that a `read()` on an empty pipe blocks the reader until the other process writes. These mechanisms are the same ones used by the shell in the previous laboratory work.